

POMDP-Based Autonomous Delivery Drone Guidance Under Wind Uncertainty

Daphne Felt

*Robotics Engineering Department
University of Colorado Boulder
Boulder, United States*

Jackson Clark

*Smead Aerospace Engineering Sciences
University of Colorado Boulder
Boulder, United States*

Abstract—This paper investigates the use of POMDP modeling and machine learning techniques to derive model-free policies for safe small UAS guidance decisions given a continuous 3D environment, limited guidance/control awareness, and unknown wind. The aircraft is represented as a 6 DOF second-order dynamics model operating under additive white Gaussian noise (AWGN) wind. We use a Kalman filter to maintain a real-time belief of the north and east wind components. The plane is rewarded for making a delivery through a small window without hitting the wall, and is penalized for any crashes. To solve this, we trained an RL model using Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) via Stable Baselines 3. The trained agents were able to learn to fly in circles to learn the wind before attempting to fly through the window. This mimics real-world information gathering maneuvers performed by human pilots in uncertain conditions. We then further explored the decision making between orbiting and delivering using an MDP in the belief space with direct ORBIT and DELIVER actions, and deriving a policy from that. Again, we were able to see the direct benefit of information gathering actions that real pilots would use in uncertain environments.

Index Terms—POMDP, Machine Learning, Unmanned Aerial Vehicle, UAS, Guidance, Navigation Control

I. INTRODUCTION

A. Aerospace Contextual Relevance

Over the past several decades, rapid advances in technology and the development of small autonomous UAS platforms have enabled a wide range of applications including package delivery, cleaning services, and infrastructure checks. These systems are extremely valuable for their hands-off employment and ability to investigate where workers may not always be able to safely access.

As these systems become incorporated in environments closer to people and property, their reasoning under uncertainty must become increasingly rigorous. Wind disturbances in particular can be large sources of error in trajectory tracking or position holds. These disturbances can cause risks to safety and mission satisfaction.

For a steady wind and a flight controller that has precise access to heading angle, with poor/noisy course angle estimates, and a precise inertial position update, it is possible for a small UAS system to autonomously

learn the inertial wind such that it can guarantee being able to fly through a bounded window region with some width tolerance. The system decides on its own when its knowledge of the wind is sufficient to fly through the bounded region given its understanding of how the wind impacts the resulting trajectory. This ability has the potential to unlock guidance support for windy variable environments where less information is available to guide aircraft specific landing locations.

This paper will discuss the example scenario in detail. It will go over problem formulation, model definition, results, and further discussions for real-world applications.

II. BACKGROUND & RELATED WORK

Our approach is founded on several core concepts within existing literature.

A. Kalman Filter Update

The Kalman filter [6] is a recursive Bayesian estimator for linear dynamical systems under Gaussian noise. The filter takes in a state and observation, performs a predict-update cycle, and outputs a new state belief of the mean and uncertainty.

Because the wind in our problem is AWGN, a Kalman filter works well as a belief update. Kalman filtering has also been applied to wind estimation for fixed-wing UAVs previously [7].

B. Markov Decision Process (MDP) and Partially Observable Markov Decision Process (POMDP)

A Markov Decision Process (MDP) [8] provides the formal framework for making decisions under uncertainty. It is defined by the tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $T(s' | s, a)$ the transition distribution, $R(s, a)$ the reward function, and $\gamma \in [0, 1)$ a discount factor. An agent seeks a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ that maximizes its expected discounted reward. Value iteration can be done to derive an optimal policy in this fully observable case.

A POMDP is an MDP on the belief space. When the agent can't directly observe the true state, the problem

is generalized to this form [9]. The POMDP adds in an observation model $Z(o \mid s', a)$. Because the true state is hidden, the agent maintains a belief, and the optimal policy maps beliefs to actions. Our problem is a POMDP because the wind vector (w_n, w_e) is not directly observed, and the agent has to infer it from position command errors.

C. Aircraft Kinematics

A fixed-wing aircraft is commonly described by a kinematic model in which the body moves at airspeed V_a along its heading ψ , with ground velocity added to by an ambient wind vector (w_n, w_e) [10]:

$$\dot{x} = V_a \cos \psi + w_n, \quad (1)$$

$$\dot{y} = V_a \sin \psi + w_e. \quad (2)$$

Altitude and heading are governed by inner-loop autopilot dynamics, which we approximate as a second-order system with damping and stiffness gains.

D. Proximal Policy Optimization and Soft Actor-Critic

Two reinforcement learning solvers are relevant to this paper, each representing a different class of decision-making algorithms under uncertainty. These methods differ primarily in how they handle policy updates, stochasticity, and partial observability.

1) *Proximal Policy Optimization (PPO)*: Proximal Policy Optimization (PPO) [11] is an on-policy policy gradient method designed to improve training stability while maintaining sample efficiency. The core idea of PPO is to constrain policy updates such that the new policy does not deviate excessively from the previous policy. This is achieved through a clipped surrogate objective, which limits the probability ratio between the new and old policies during optimization. PPO was designed as a simpler and more general alternative to Trust Region Policy Optimization [13], which had hard KL-divergence constraints on policy updates.

By restricting large policy updates, PPO avoids the instability associated with traditional policy gradient methods while still allowing for iterative improvement. PPO is well-suited for continuous control problems and was applied in this work to learn policies over discrete action spaces.

2) *Soft Actor-Critic (SAC)*: Soft Actor-Critic (SAC) [14] is an off-policy actor-critic algorithm that incorporates entropy maximization into the reinforcement learning objective. In addition to maximizing expected reward, SAC encourages exploration by maximizing the entropy of the policy. This results in more stochastic policies that are less prone to premature convergence to suboptimal solutions.

SAC uses a learned Q-function and a stochastic policy, updating both using off-policy data. The inclusion of entropy regularization provides a balance between exploration and exploitation, making SAC particularly effective in continuous and high-dimensional action spaces.

III. PROBLEM FORMULATION

The scenario used to evaluate the question at hand involves a package delivery fixed-wing drone that is tasked with delivering a package in an environment with a constant unknown wind gust. To deliver the package, the drone must successfully fly through a window of fixed width and depth to reach a delivery region. Onboard the aircraft are sensors that allow for precise inertial updates and heading updates, but poor inertial velocity measurements. The aircraft's flight controller includes a wind estimate in its Kalman belief update, and it uses the difference between the expected position (with no inertial wind) and the true position estimated after some time step dt to make observations. The expected position is calculated using known 2nd-order dynamics approximations of the inner-loop controller's step response. These dynamics also happen to be the true dynamics of the aircraft in this formulation, but with an additive white gaussian noise component representing process noise. To successfully fly through the window, the drone must have a precise enough belief of the wind such that the heading angle it chooses to use based on the wind does not result in a collision or crash. Intuitively, a heuristic policy would be to fly around until the estimate of the wind is precise enough, and then fly through the window. Two separate models of varying fidelity are used in this paper jointly to find the true optimal policy for the complex problem proposed above.

A. The Continuous POMDP Model

The UAV guidance problem can be broken down into a POMDP model as follows:

1) *States*: The state has 8 continuous degrees of freedom including 3D position, heading angle, rate of change of height, and rate of change of heading angle. The state also includes the current wind in the environment. The physical bounds for this problem span 200m by 200m by 200m in length.

$$x = [p_n, p_e, h, \dot{h}, \psi, \dot{\psi}, w_n, w_e]^T \quad (3)$$

2) *Actions*: The action space has 2 continuous dimensions consisting of commanded height and commanded heading angle.

$$u = [h_c, \psi_c]^T \quad (4)$$

3) *Transitions*: The transitions between states are deterministic and obey the following 2nd-order kinematic equations:

$$\dot{x} = \begin{bmatrix} V_a \cos(\psi) + w_n \\ V_a \sin(\psi) + w_e \\ \dot{h} \\ -b_h \dot{h} + b_h(h_c - h) \\ \dot{\psi} \\ -b_\psi \dot{\psi} + b_\psi(\psi_c - \psi) \\ 0 \\ 0 \end{bmatrix} \quad (5)$$

where

$$[b_h \quad b_h \quad b_\psi \quad b_\psi] \quad (6)$$

are constant parameters that define the unique behavior of the 2nd-order system.

Inside the integration function that propagates these states forward under an action, an additional rate limiter is present to cap the rate of change of height to 30 m/s, and heading angle to a max rate of 100 deg/s.

4) *Reward Structure*: The reward structure depends solely on the position of the drone. It is rewarded for flying through the window and penalized for hitting obstacles/going out of bounds. There is also a small penalty on time to encourage faster deliveries.

$$R(s, a, s') = \begin{cases} -100, & s' \in \mathcal{W} \\ 10, & s' \in \mathcal{T} \\ -0.25, & \text{otherwise} \end{cases} \quad (7)$$

where $\mathcal{W}$ represents continuous position states on or inside of the wall and $\mathcal{T}$ represents continuous position states that reach the exiting side of the window. Note that the collision and delivery states are absorbing states and the model ends once one of these states are reached. Also note that the -0.25 step reward is really a step reward over 1 second, and for varying time step sizes this reward would be scaled accordingly. The target state (window) is an extruded square with fixed square planar x and z lengths placed at the same location y as the wall. The depth of the wall and the window are the same. The wall is placed at $y = 150\text{m}$ with a width of 20 – 40m (we experiment with both sizes) and the window is placed at $x = 100\text{m}$ and $z = 100\text{m}$.

5) *Initial Conditions*: The aircraft has a starting uniformly distributed random starting state of the following:

$$x_0 = [100, \quad 0, \quad 100, \quad 0, \quad 90, \quad 0, \quad 4, \quad 1]^T \quad (8)$$

with the following $\pm$ values that define the range of the uniform distribution:

$$\Delta x_0 = [50, \quad 50, \quad 50, \quad 0, \quad 90, \quad 0, \quad 2, \quad 2]^T \quad (9)$$

We specifically let the wind vector change with the start of each problem to ensure the agent learns per episode, instead of learning an optimal policy throughout episodes for one constant wind state.

6) *Observations & Beliefs*: As briefly discussed before, the partially unobservable component of this formulation comes from the wind, whose belief state is maintained with a Kalman Filter belief update. The belief of the wind is normally distributed with mean μ and covariance Σ defined by:

$$b_t(x) = \mathcal{N}(x_t \mid \mu_t, \Sigma_t) \quad (10)$$

Since the belief for a POMDP is fundamentally based on the series of past actions and observations,

$$b_t(x) = \mathbb{P}(x_t = x \mid o_{1:t}, a_{1:t-1}) \quad (11)$$

for a new action and observation there must be a new belief

$$b_{t+1} = \tau(b_t, a_t, o_{t+1}) \quad (12)$$

where this belief update function is the Kalman filter update itself. The observation/measurement is the difference between the expected state, $x_{predict}$ & $y_{predict}$ with no wind and the observed state, x_{true} & y_{true} (with wind), plus some room for process noise not modeled by the predicted dynamics function R . This, in theory, is the wind vector itself.

$$y_{t+1} = x_{t+1}^{meas} - x_{t+1}^{predict} \quad (13)$$

This difference is fed into a Kalman filter belief update, which, for a fixed true state estimator, is just a recursive linear least squares problem:

$$K_{t+1} = \Sigma_{b,t} H^T (H \Sigma_{b,t} H^T + R)^{-1} \quad (14)$$

$$\mu_{b,t+1} = \mu_{b,t} + K_{t+1} y_{b,t+1} \quad (15)$$

$$\Sigma_{b,t+1} = (I - K_{t+1} H) \Sigma_{b,t} \quad (16)$$

The belief update can be seen in Fig 1.

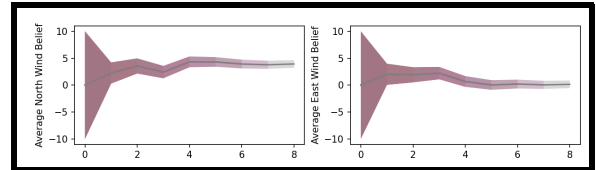


Fig. 1: Wind belief update over 8 actions. Mean line in center with 1 std bounds shaded

The mean and variance (no covariance is used since the wind components are implemented independent of one another) are then fed into the observation after each action step in the POMDP.

B. The Discrete MDP Model

As one of our later solution methods, we abstract out this POMDP into an MDP. This can be done because a POMDP is an MDP on the belief space. With a Kalman filter belief update on the observations, the belief is fully summarized by (μ, Σ) . Because any average known wind can be corrected for, only the wind uncertainty matters for delivery risk. The observation that drives the high level policy is therefore deterministically updated by the Kalman filter with each step. This collapses the POMDP to a fully observed MDP whose only state variable is σ_n .

We use all of the following information to fully define the MDP.

1) *States*: σ discretized into $N_\sigma = 15$ logarithmic spaced bins between 0.3 and 15 m/s, plus a terminal state

2) *Actions*: {ORBIT, DELIVER}.

3) *Transitions*: DELIVER goes to the terminal state with probability 1. ORBIT maps σ to the bin containing the value given by Eq. (22).

4) *Rewards*: ORBIT costs $T_{\text{orbit}} \cdot r_{\text{step}} = -2.25$. DELIVER returns the expected reward $p_{\text{deliver}}(\sigma) r_{\text{deliver}} + [1 - p_{\text{deliver}}(\sigma)] r_{\text{crash}}$.

5) *Discount*: $\gamma = 0.95$.

IV. SOLUTION APPROACH

A. Python, Gymnasium, Stable-Baselines3, TensorBoard, Julia

For training the PODMP model, Python was selected as the preferred programming language to carry out computation for its ease-of use and familiarity by the authors and also its integrated libraries that naturally support reinforcement learning (RL) techniques. These libraries are Gymnasium and Stable-Baselines3. The model state bounds, transition dynamics, and reward functions were encoded into a class that was wrapped in a Gymnasium object with "standard" episodic simulator functions that Stable-Baselines3 could then interact with and learn from. These wrapper functions include initialization, an environment reset, and a step function that sets the new state, returns reward, a terminated flag, a truncated flag, and a debugging dictionary from the step.

TensorBoard was used to visualize the reinforcement learning training stability and performance, while the converged policy was evaluated back in Python using the base model class using a common python plotting library called Matplotlib.

To solve the MDP with value iteration, we also used Julia. We output a csv policy that could be read in through our python environment for use.

B. Reinforcement Learning Training

Due to the natural support of Stable-Baselines3 for the Gymnasium environment, it was very straightforward to implement training. A simple definition of a model object (for PPO and SAC) was declared and then the method `.learn()` was used to train the model. Each model had varying levels of parameter fidelity available to them, and each model naturally found success with different action space types.

1) *PPO*: PPO was the starting RL algorithm used here, and for simplicity it was set up with a discrete action space of

$$h_{c,PPO} = [0, 50, 100, 150, 200] \text{ (m)} \quad (17)$$

$$\psi_{c,PPO} = [-180, -90, 0, 90, 180] \text{ (deg)} \quad (18)$$

We trained the model using a learning rate of 0.0001 with Stable-Baselines3.

2) *SAC*: The evaluation of SAC to solve the problem was inspired by several other papers [1] [2] [5] that used SAC for wind-learning algorithms. SAC has the ability to exploit continuous action spaces, making its optimal policy much more flexible. It's action space was set by bounds instead of discrete points:

$$h_{c,SAC,limits} = [0, 200] \text{ (m)} \quad (19)$$

$$\psi_{c,SAC,limits} = [-180, 180] \text{ (deg)} \quad (20)$$

C. MDP Solution

1) *Flight Controller Design*: For the MDP formulation, we abstracted out the low level actions by providing a high-level controller with only two actions: ORBIT and DELIVER:

$$u_{MDP} = [ORBIT, DELIVER] \quad (21)$$

- **ORBIT**. Fly one closed loop ($T_{\text{orbit}} = 9$ s) by stepping through a fixed list of nine ground-track headings forming a circle. Each step the altitude command is set to z_w and the heading is wind-corrected. One ORBIT yields T_{orbit} wind observations, so the Kalman variance shrinks via the information form

$$\frac{1}{\sigma_{\text{new}}^2} = \frac{1}{\sigma^2} + \frac{T_{\text{orbit}}}{R_{KF}}, \quad R_{KF} = \sigma_w^2. \quad (22)$$

- **DELIVER**. Aim straight at $(x_{\text{window}}, y_{\text{wall}})$ with wind-corrected heading and altitude command z_w , fly until $y \geq y_{\text{wall}}$. Success or failure depends on the residual wind error.

Both the orbit and delivery controllers use wind correction. Given a desired ground-track heading ψ_{des} and a wind belief μ , this returns the air heading ψ_{air} to be commanded. Setting the cross-product of the desired ground vector and the (air + wind) ground vector to zero gives:

$$\sin(\psi_{\text{des}} - \psi_{\text{air}}) = \frac{\mu_e \cos \psi_{\text{des}} - \mu_n \sin \psi_{\text{des}}}{V_a}, \quad (23)$$

so the air heading is $\psi_{\text{air}} = \psi_{\text{des}} - \delta$ with $\delta = \arcsin(\dots)$.

With this controller, the MDP focuses on the tradeoff between reducing risk with information gathering vs delivering as fast as possible instead of trying to learn low level flight dynamics.

2) *Delivery success probability*: To help the MDP solve for expected reward, we need to calculate the success probability for the DELIVER action.

If the agent corrects its heading using μ , the residual lateral wind error is $w_n - \mu_n \sim \mathcal{N}(0, \sigma^2)$. Over the approach distance $d = y_{\text{wall}} - y_0$, this drifts the aircraft sideways by $(w_n - \mu_n) d / V_a$. The plane won't crash when this drift stays inside the window half-width, giving the tolerance

$$w_{\text{tol}} = \frac{w_{\text{half}} \cdot V_a}{d} \quad (24)$$

The success probability as a function of the current uncertainty is then

$$p_{\text{deliver}}(\sigma) = \Pr(|X| < w_{\text{tol}}) \mid X \sim \mathcal{N}(0, \sigma^2) \quad (25)$$

This is used as mentioned in the MDP formulation to calculate average expected reward.

V. RESULTS

A. Reinforcement Learning Solvers

1) *PPO*: PPO policies delivered successfully around 50% of the time for large, 40m window widths, and the model saw learning throughout training. The policy favored circling several times before eventually attempting delivery. It took about 20 minutes to run for 200k steps, and reached a mean episode reward of around -50. The training log is shown in Figure 2.

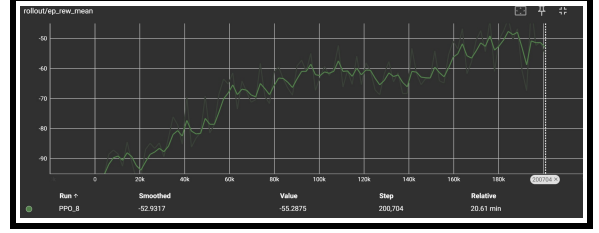


Fig. 2: PPO Training Log.

Figure 3. is an example of a successful PPO policy rollout for a 40m window width (the wind vector is drawn from the s0 state in blue).

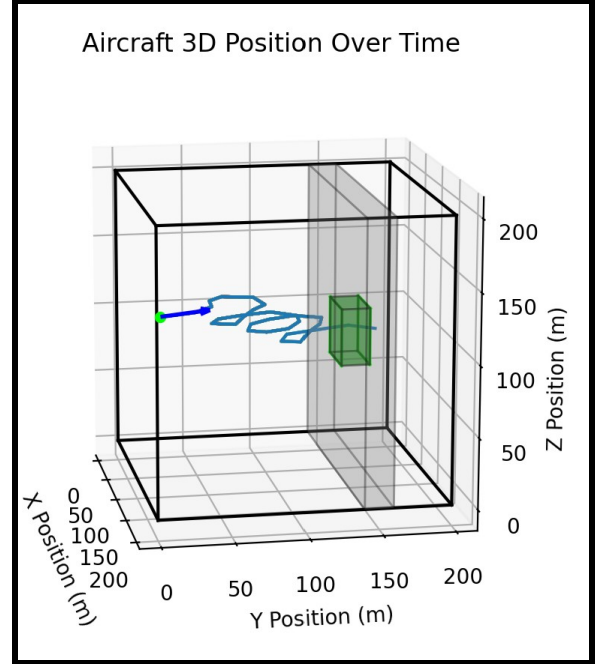


Fig. 3: PPO Policy Rollout Simulation.

2) *SAC*: The training for SAC went surprisingly smooth given the massive action space available. The agent was able to converge to a solution that assured flying through the target window from anywhere in the starting probability distribution bounds. The convergence took around 25 minutes of training for 80k steps and reached a mean reward of 8.5. The training log is shown in Figure 4 below.

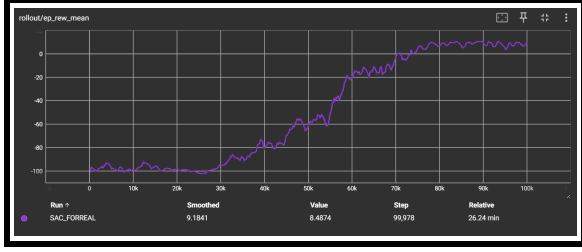


Fig. 4: SAC Training Log.

This implemented and visualized in 3D space is shown in Figure 5. Note that the fidelity of the actions allowed the agent to fly through a window of 20m in this case without doing a complete orbit first:

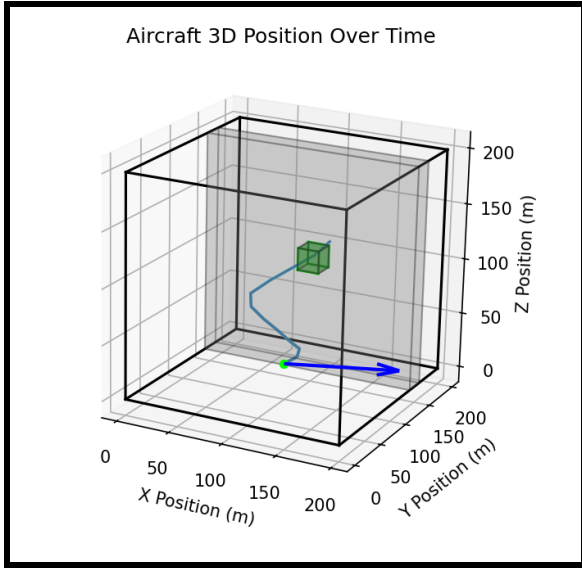


Fig. 5: SAC Policy Rollout Simulation.

3) *MDP*: For the MDP formulation, we solve the MDP with value iteration to find the optimal policy for each wind uncertainty bin. This policy changes depending on window width.

With a window width of 40m, it can be seen in Figure 6 that the policy orbits once before delivering.

Cutting the window width in half to 20m, the policy waits for a much higher certainty before attempting to deliver the package. This results in the agent taking two orbits before attempting delivery as shown in Figure 7.

VI. DISCUSSION

The general trend of these methods points to support the general heuristic policy noted in the problem introduction. Each analysis method shows periods of information gathering actions by the delivery plane to reduce its uncertainty in the wind such that it is able to successfully guide the plane through the window. It can

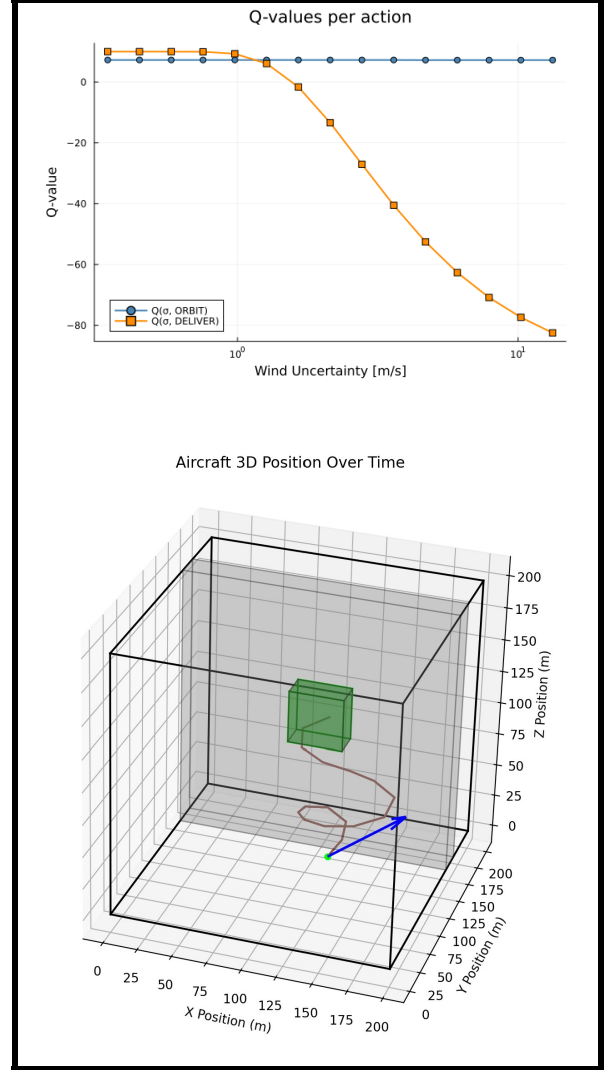


Fig. 6: MDP Q-Values Over Belief & MDP Policy Simulation Rollout w/ 40m Window.

be seen that the MDP and SAC solutions both prove to be an equally viable solution via their empirical success probabilities of employing their own optimal policies.

The implementation of RL algorithms came not without learning moments for the authors related to reward shaping to speed up initial learning of the window location and nudging the aircraft by giving some higher rewards for smaller uncertainties. It was found that RL agents will exploit small rewards indefinitely if it means they accumulate infinite reward, and so the reward shaping had to be done very carefully to not change the true optimal policy. Learning rate tuning was also slightly variable, but its impact on convergence and "rubber-banding" of the learning curves was obvious between even small changes.

Overall, the results show theoretical proof of utiliz-

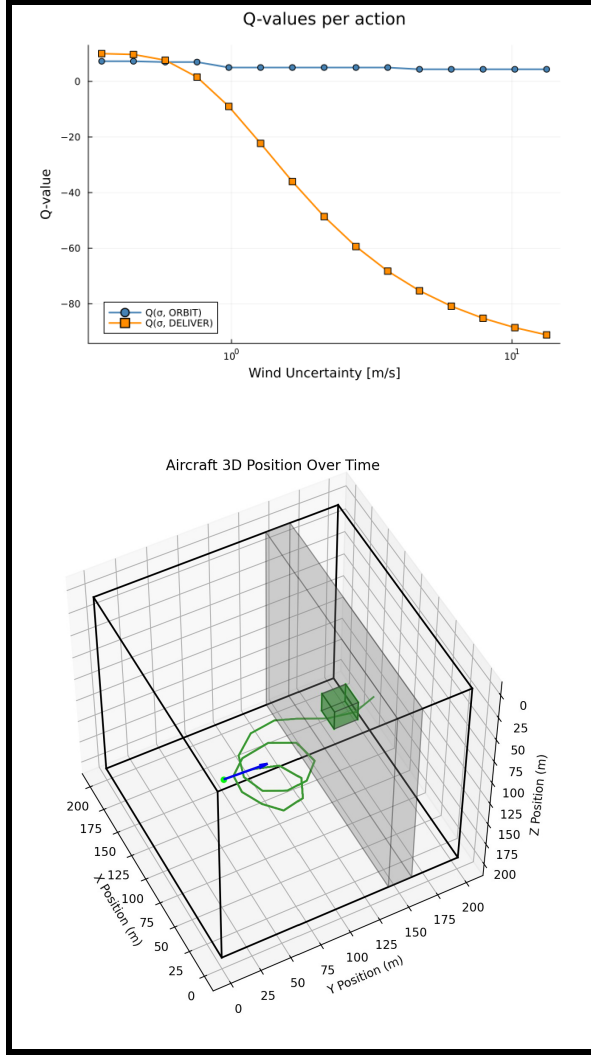


Fig. 7: MDP Q-Values Over Belief & MDP Policy Simulation Rollout w/ 20m Window.

ing wind belief updates to improve guidance decisions through reinforcement learning for small UAS aircraft. It can be (rather obviously) noted that the smallest window width for successful training of the agents in this study across all methods was 20m in width, which is 20x the wingspan of the average small UAS aircraft. And so, there is definitely room for improvement in the accuracy that the RL agent can converge to with superior RL training abilities.

A potential critique of this paper for real applications on the wind belief front would be that the wind belief update employed runs on the assumption that AWGN noise was the primary noise, and was consistently distributed for all time. This is because in real applications there exists a stochastic and/or spectral element to non-gusting wind disturbances and/or turbulence. However, it

could also be possible to learn the spectral frequency of the wind for the environment [1] and include its impact on sampled measurements of the wind (that include gusts and turbulence) to create additional robustness and quicker convergence to learn the primary gusting component of the wind in real applications.

VII. CONCLUSION

While all three strategies for solving the UAV were successful to some extent, each of their strengths and weaknesses can be highlighted in the table below.

	PPO	MDP	SAC
Action space	Discrete	Continuous	Continuous
Success at width 20 m	13/100	45/100	60/100
Success at width 40 m	56/100	92/100	94/100
Avg Episode Reward	-53.12	-6.14	-5.78
Runtime (min)	20	<1	26
Iterations	200k	500	100k

TABLE I: Comparison of algorithms for solving UAV POMDP.

PPO generally performed worse at delivery and average episode reward, while MDP and SAC were both highly successful. The main differences between these two methods were actually in the runtime to solve the policy. Because MDP simplifies the problem to such a high extent, the time was much lower than other methods.

Overall, we learned a valuable lesson in that simplifying POMDPs to MDPs can drastically lower computational complexity while still delivering highly viable solutions. While not all problems can be simplified in this way, it's an incredibly valuable tool for certain cases.

VIII. CONTRIBUTIONS AND RELEASE

A. Author Contributions

Daphne Felt constructed the state space, reward functions, and Kalman belief update for the underlying environment. She also wrapped the POMDP to be compatible with the Gymnasium library. She undertook the problem of transforming the POMDP into an MDP with controller and solving it to create a policy. She also helped train the PPO model and benchmark. For the paper, Daphne wrote up the Abstract, Background and Related Work, Discrete MDP Model, PPO and MDP Results, and Conclusion.

Jackson Clark developed the action space and the transition dynamics function for the underlying environment and worked to train RL agents on the environment using PPO and SAC. His contributions to the report mainly included the Introduction, RL Algorithm definitions, the POMDP definition, the Solution Approach section up until the MDP solution, as well as SAC results and Discussion.

The authors grant permission for this report to be posted publicly.

Core algorithms implemented were the continuous POMDP transition dynamics function, the POMDP belief update calculation, the continuous reward function, and the guidance controller for both actions orbit and delivery in the the MDP formulation.

REFERENCES

- [1] Q. Ma, Y. Wu, M. U. Shoukat, Y. Yan, J. Wang, L. Yang, F. Yan, and L. Yan, "Deep reinforcement learning-based wind disturbance rejection control strategy for UAV," *Drones*, vol. 8, no. 11, p. 632, 2024, doi: 10.3390/drones8110632
- [2] J. Amendola, L. R. Cenkeramaddi, and A. Jha, "Drone landing and reinforcement learning: State-of-the-art, challenges, and opportunities," **IEEE Open Journal of Intelligent Transportation Systems**, 2024, doi: 10.1109/OJITS.2024.3444487.
- [3] J. Wilson and R. Riccardi, "Enabling intelligent onboard guidance, navigation and control using reinforcement learning," in *Proc. AIAA SciTech Forum*, 2022. [Online]. Available: <https://strathprints.strath.ac.uk/81480/>
- [4] M. Ran, J. Li, and L. Xie, "Reinforcement learning-based disturbance rejection control for uncertain nonlinear systems," *arXiv:2212.03828*, 2022. [Online]. Available: <https://arxiv.org/abs/2212.03828>
- [5] H. Weber, "Autonomous drone navigation using reinforcement learning algorithms," **Scientific Journal of Artificial Intelligence and Blockchain Technologies**, vol. 2, no. 3, pp. 17–25, 2025, doi: 10.63345
- [6] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [7] S. Sun, K. Dong, C. Guo *et al.*, "A wind estimation based on unscented Kalman filter for standoff target tracking using a fixed-wing UAV," *International Journal of Aeronautical and Space Sciences*, vol. 22, pp. 366–375, 2021, doi: 10.1007/s42405-020-00290-7.
- [8] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Hoboken, NJ: Wiley, 1994.
- [9] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.
- [10] P. Chen, G. Zhang, J. Li, Z. Chang, and Q. Yan, "Path-following control of small fixed-wing UAVs under wind disturbance," *Drones*, vol. 7, no. 4, p. 253, 2023, doi: 10.3390/drones7040253.
- [11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.
- [12] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [13] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. Int. Conf. Machine Learning (ICML)*, pp. 1889–1897, 2015.
- [14] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Machine Learning (ICML)*, vol. 80, pp. 1861–1870, 2018.